ARTEX and Agentic Offensive Frameworks: A Detection Engineering Analysis of Machine-Speed Intrusion Activity

Abstract

Between late August 2026 and October 2026, Google Threat Intelligence Group and Mandiant observed multiple threat actors weaponising open-source, multi-agent pentesting frameworks, principally ARTEX, to conduct highly automated intrusions across the attack lifecycle. ARTEX, originally developed by the GitHub user Autumn-27 and written in Go with an embedded Next.js web console, abstracts the attack lifecycle across six specialised sub-agents that leverage APIs from frontier and open source models to translate high-level tactical goals into actionable machine commands. The intrusions are notable not for novel techniques but for their compressed execution velocity. The underlying mechanics, including compromised credentials, SQL injection, Kerberoasting, DCSync and Pass-the-Hash, remain standard adversary tradecraft. The defensive challenge lies in detecting the execution pattern and velocity rather than the individual techniques. This working paper maps the observable behavioural patterns documented in the source to four Sigma detection rules covering probe file creation, account creation with ArtexProbe naming conventions, rapid webshell deployment via MySQL general log poisoning, and high-volume web directory fuzzing. It also identifies what remains invisible to defenders, including the LLM orchestration layer itself, and notes the limitations of detections built on burst timing when adversaries may soon trade velocity for stealth.

What happened

On 30 October 2026, Google Threat Intelligence Group and Mandiant published findings on a class of autonomous, multi-agent LLM-driven offensive frameworks that have crossed from experimental use to in-the-wild intrusion activity. The report, authored by Aurora Blum, David Mainor, Rohit Nambiar, Omar ElAhdan, Matthew McWhirt, Will Silverstone, Bhavesh Dhake and Chris Busse, documents observations from Mandiant incident response engagements dating from late August 2026 onward. The principal framework examined is ARTEX, though the report also covers PentAGI, CAIRN and Hexstrike, as well as potentially custom agentic harnesses.

ARTEX is described as an autonomous, multi-agent LLM-driven pentesting and offensive command-and-control platform written in Go with an embedded Next.js web console. The original repository was developed by the user Autumn-27 and released on GitHub. Although the original repository was taken down by its creator in October 2026 due to widespread threat actor abuse, the framework continues to proliferate globally via community forks, including localised variations such as ScopeWeaver and a commercial version dubbed ARTEX PRO.

The source establishes that ARTEX has been adopted by a diverse range of threat actors across the motivational spectrum, including financially motivated extortion groups and suspected cyber espionage operators. GTIG tracks one campaign associated with ARTEX as UNC7228, which overlaps with activity tracked by Microsoft as the PRC-nexus actor STORM-0940. Wiz Research identified targeting, compromise and data theft affecting multiple entities in the US and Europe across the enterprise software, industrial automation, semiconductor manufacturing and design, and telecommunications industry verticals. This actor moved extremely quickly, typically moving from first compromised account to deep system access in around four hours and in one case as rapidly as 15 minutes.

The source is explicit about the nature of the defensive challenge. The authors state: “While the

mechanics of the intrusions remain the same, such as using compromised credentials and targeting unpatched systems, the speed and scale at which frameworks can exploit these known weaknesses introduces a new defensive challenge." They further note: "At the time of reporting, we assess that the current adversary use of autonomous offensive frameworks is still in early stages characterized by open source tools currently being used in low-sophisticated ways, resulting in broad and noisy, but still successful campaigns." The authors also observe: "Frameworks such as ARTEX compress attack timelines to machine speed; however, they currently trade stealth for velocity."

In several cases documented in the report, adversaries provided ARTEX with compromised credentials for the targeted organisation and instructed the framework to execute subsequent stages of the attack lifecycle. The framework then autonomously executed actions across multiple phases, including initial reconnaissance, establishing footholds, privilege escalation, internal reconnaissance, lateral movement, maintaining presence and completing the mission.

During initial reconnaissance, ARTEX automatically mapped the effective permissions of obtained Azure credentials, including SAS tokens, Service Principals, Kudu credentials and Storage Account keys, using sub-second capability checks. The source documents ARTEX writing and immediately deleting a tiny test blob in approximately 330 to 400 milliseconds and running an 8-second ARM deployments and validate dry-run sweep across 11 Azure resource types. In AWS environments, ARTEX tested compromised IAM user permissions using DryRun flags and placeholder IDs to evaluate EC2 compute, EBS volume and snapshot capabilities. The source records 27 events in 23 seconds of active burst time performing rapid probing of AWS permissions across different security groups. Over a span of more than 500 web requests, ARTEX probed various file names and extensions across web directories, attempting to fuzz which URL endpoints the web server would allow it to access based on HTTP response codes of 200 or 403.

To establish a foothold, ARTEX abused Kudu administrative APIs to execute arbitrary commands and stage custom Python, C# and malicious PowerShell scripts. It established command and control on a victim's internal Windows database server by using native curl to pull down dynamically generated PowerShell and batch scripts and post back command results across 491 rotating webhook.site URLs.

For privilege escalation, ARTEX attempted to launch cloud compute instances with potentially high-value IAM roles, though these attempts failed due to invalid instance profile ARNs. It then escalated privileges by running Kerberoasting scripts to harvest service tickets, using SQL injection to dump database credentials, and following an offline cracking pause, executing DCSync scripts to steal the domain krbtgt key, cloud AD-sync credentials and Domain Controller machine hashes. ARTEX created accounts with is_admin set to true within minutes, identifying and utilising a writable admin flag.

The source provides a detailed account of one technique that illustrates the framework's iterative trial-and-error approach. In an attempt to pivot from a database-execution context to a system-execution context, ARTEX used an application-specific SQL API to modify a MySQL instance to write log files with a .jsp extension to publicly accessible locations. The framework ran SET GLOBAL general_log_file followed by SET GLOBAL general_log = 1, then ran SQL SELECT statements in the form of SELECT with webshell content enclosed in JSP scriptlet tags. Due to how the underlying Java web servlet handles JSP files, the Java code was parsed and compiled into the process memory space, providing a means of execution. Log activity shows multiple iterations of this sequence, with ephemeral, somewhat-incremental file names including cl1217.jsp, f1219a.jsp and z1219.jsp for each webshell candidate, each one executing a different command. Errors such as

a variable already being defined caused webshells to fail, and ARTEX likely refactored its approach, writing several unique webshells, each with a different goal, some correcting previous errors until the JSP webshells executed cleanly and returned valid command output.

During access-probing efforts, ARTEX generated files of various extension types with the filename pattern artex-probe- followed by five-digit somewhat-incremental numbers. The source notes that based on available visibility, it is difficult to comment on the exact significance of these numbers or how they increment. However, ARTEX was seen using similar incremental numeric markers within dropped file names, created account names, their passwords and within email subjects.

For persistence, ARTEX generated a new SSH key pair, allocated a dedicated public Elastic IP, attempted to create a custom security group but failed due to permission issues, and launched persistent t3.micro instances bound to existing security groups to establish an externally addressable jump host and internal foothold. It gained persistence through account takeover by resetting passwords for identified users, creating multiple accounts with plaintext passwords which included ArtexProbe in the naming convention, and creating administrator accounts.

In the mission completion phase, ARTEX interrogated backend SQL databases, extracting approximately 400 KB of query results across roughly 20 queries, and performed mass blob reads. It deployed custom boot scripts via ephemeral, keyless EC2 instances to potentially harvest data from multiple mounted EBS snapshots and volumes. It enumerated over 110,000 Active Directory objects from a domain controller and retrieved the Domain Admins group, sending command output to rotating webhook.site endpoints via curl. ARTEX created a new 100 GB volume and expanded the root disk of launched instances to 60 GB, performing preparatory steps likely intended for exfiltration via automated collection scripts.

The source also documents an abuse of legitimate application features. Rather than executing complex malware, ARTEX weaponised built-in application features, forcing an application's notification system to broadcast test emails to over 1,300 internal users, turning a trusted notification channel into a mass internal messaging tool. It then dropped to single-recipient test emails rather than the mass emailing. In one case, ARTEX edited an employee's record in a legacy database, created and deleted notes and status rows, and pointed checklist responses at its own probe file.

The report describes a notable observation regarding concurrent agent behaviour. In one instance, ARTEX accessed an external-facing Linux Weaver OA web server to execute SQL commands via exposed API calls. This process generated 240 log events and exhibited errors potentially caused by multiple sub-agents overwriting one another while pursuing the same objective.

Beyond ARTEX, the source documents in-the-wild abuse of PentAGI, developed by VXControl and offered in open source and commercial editions. PentAGI is designed so that the user only needs to describe a target and a goal, enabling entry-level users to run complex cybersecurity campaigns. The framework uses a team of specialised agents that break tasks into sub-tasks called Flows, execute each step and report on it. Tools run in isolated sandboxes running Kali Linux rather than in the host environment. The majority of observed PentAGI activity remained at the stage of WHOIS asset mapping or active vulnerability scanning, though analysis of operational artifacts indicates that some operations progressed to active intrusions and persistence across multiple organisations. In one case, a customised attack orchestration platform built on the open-source version of PentAGI was likely used between at least late August and early October 2026 to conduct systematic reconnaissance and vulnerability scanning against organisations in Asia, including system integrators and major telecommunications firms. In another instance, PentAGI was chained with proxy services to target the aerospace sector.

The source situates these observations within a broader evolutionary framework. GTIG tracks the evolution of LLM augmentation in cyber operations across multiple stages. Stage 1, from the end of 2025 to early 2026, involved prompt-based use of LLMs for writing and debugging code, drafting phishing emails, creating personas and conducting light reconnaissance. Stage 2, in early 2026, involved embedding LLM capabilities into malware, with adversaries trusting LLMs to take low-failure actions such as obfuscating payloads. Stage 3 involved incorporating LLM-enabled automation into operations for discrete tasks. The source text is truncated at Stage 3, and the full description of subsequent stages is not available in the provided material.

Detection opportunities

The source provides several concrete, observable behavioural patterns that defenders can monitor. Each detection opportunity below is tied to a Sigma rule file in the detection pack.

The first detection opportunity addresses the artex-probe file naming pattern. During access-probing efforts, ARTEX generated files of various extension types with the filename prefix artex-probe- followed by five-digit somewhat-incremental numbers. The source notes that ARTEX used similar incremental numeric markers within dropped file names, created account names, passwords and email subjects. Defenders can watch for file creation events matching this naming convention in filesystem audit logs, EDR file creation events and web server logs. This detection is codified in 01-artex-framework-probe-file-creation.yml. The rule targets the T1083 technique for File and Directory Discovery, as the probe files are generated systematically to test write access and URL endpoint availability. This is an alert-type detection because the naming convention is highly specific to ARTEX and carries low risk of false positives from legitimate administrative activity.

The second detection opportunity targets account creation using the ArtexProbe naming convention. ARTEX created multiple accounts with plaintext passwords that included ArtexProbe in the naming convention, and created administrator accounts with is_admin set to true within minutes. Defenders can monitor directory service logs, including Active Directory and cloud IAM, as well as EDR telemetry, for new account creation events where the account name or password contains the string ArtexProbe. This detection is codified in 02-artex-framework-account-creation-with-artexprobe-n.yml. The rule targets the T1136 technique for Create Account. This is an alert-type detection because the presence of ArtexProbe in account naming is a strong indicator of ARTEX framework activity, specifically its persistence and privilege escalation logic that programmatically generates account names using its own probe naming convention.

The third detection opportunity addresses rapid iterative webshell deployment via MySQL general log poisoning. The source documents a specific sequence: ARTEX ran SET GLOBAL general_log_file followed by SET GLOBAL general_log = 1, then ran SQL SELECT statements containing JSP webshell content enclosed in scriptlet tags. This sequence was repeated multiple times with ephemeral, incremental filenames including cl1217.jsp, f1219a.jsp and z1219.jsp. Errors caused ARTEX to refactor its approach, writing several unique webshells until they executed cleanly. Defenders can monitor MySQL query logs, web server access logs and filesystem monitoring for this sequential pattern of general log manipulation followed by SELECT statements containing webshell content. This detection is codified in 03-artex-rapid-webshell-deployment-via-mysql-general.yml. The rule targets T1505.003 for Server Software Component: Web Shell. This is an alert-type detection because the combination of general log file path manipulation targeting web-accessible directories and SELECT statements containing JSP scriptlet tags is a strong indicator of log poisoning for webshell deployment, regardless of whether ARTEX is the actor.

The fourth detection opportunity targets high-volume web directory fuzzing. The source documents over 500 web requests probing various file names and extensions across web directories, with ARTEX evaluating HTTP 200 versus 403 response codes to determine accessible paths. Defenders can monitor web server access logs and WAF logs for high volumes of requests probing different file names and extensions across web directories within a compressed timeframe. This detection is codified in 04-high-volume-web-directory-fuzzing-indicative-of-ag.yml. The rule targets T1595.003 for Active Scanning: Wordlist Scanning. This is a hunt-type detection because high-volume web directory enumeration can also originate from legitimate vulnerability scanners, red team tools or automated testing frameworks. The distinguishing factor for agentic framework activity is the volume and pattern consistency consistent with automated enumeration rather than manual browsing.

Several additional detection opportunities are documented in the assessment but do not have corresponding Sigma rules in the provided pack. Compressed burst timing in authentication and API activity is observable as 27 events in 23 seconds of active burst time, sub-second capability checks with 330 to 400 millisecond test blob write and delete cycles, and 8-second ARM validation sweeps across 11 resource types. This is a hunt-type detection requiring cloud audit logs from Azure Activity or AWS CloudTrail. Rapid AWS permission probing across security groups uses DryRun flags and placeholder IDs to evaluate EC2, EBS and snapshot capabilities, observable in AWS CloudTrail. Rotating webhook.site URLs for command and control via native curl involves 491 rotating URLs used to pull down dynamically generated PowerShell and batch scripts and post back command results, observable in DNS logs, proxy logs and EDR process execution logs. Kerberoasting, DCSync and Pass-the-Ticket executed in a compressed timeframe chains credential theft and lateral movement techniques autonomously, observable in Active Directory security logs, EDR and Windows Event Logs. Ephemeral keyless EC2 instances with custom boot scripts for snapshot harvesting are observable in AWS CloudTrail and EDR on cloud instances. Mass internal email broadcast via an application notification system forced over 1,300 internal users to receive test emails followed by a drop to single-recipient test emails, observable in mail server logs and application logs. These patterns are documented in the source and assessment but are not covered by the four Sigma rules provided.

What cannot be detected

Several aspects of ARTEX and similar agentic offensive frameworks are invisible to defenders operating within the victim environment.

The LLM orchestration layer itself is entirely opaque to the victim. The source describes frameworks calling APIs from frontier and open source models, but those calls occur between the attacker's framework and the LLM provider, not within the victim's infrastructure. The victim organisation has no visibility into which model is being used, what prompts are being sent, what goals have been specified or how the framework is decomposing tasks across its sub-agents. The six specialised sub-agents that ARTEX uses to abstract the attack lifecycle operate outside the victim's telemetry surface. The defender sees only the downstream actions, the commands executed, the files created and the network connections made, not the reasoning or orchestration that produced them.

Valid credentials used successfully by the framework are indistinguishable from legitimate use at the authentication event level. The source states that in several cases, adversaries provided ARTEX with compromised credentials for the targeted organisation. When ARTEX authenticates using those credentials, the authentication event itself appears normal. The source notes that the underlying mechanics remain the same, meaning the individual techniques are not novel. Only the

execution velocity and pattern differ, and those differences may not be visible at the individual event level. A single API call made by ARTEX looks identical to the same call made by a legitimate administrator or a human adversary.

Custom agentic harnesses with no public signatures would not be identifiable by tool-specific indicators. The artex-probe file naming pattern and the ArtexProbe account naming convention are specific to ARTEX. The source mentions other frameworks including PentAGI, CAIRN and Hexstrike, as well as potentially custom agentic harnesses. These other frameworks would not produce the same naming conventions. A custom harness built by a sophisticated actor would leave no tool-specific fingerprints at all. The defender would need to rely entirely on behavioural patterns such as burst timing and request volume, which the source explicitly warns may degrade as adversaries improve their operational security.

The offline cracking pause documented in the source represents another detection gap. The source describes ARTEX running Kerberoasting scripts, using SQL injection to dump database credentials, then following an offline cracking pause before executing DCSync scripts. The offline cracking occurs entirely outside the victim environment. The defender observes the Kerberoasting activity and later observes the DCSync activity, but the intervening credential cracking happens on the attacker's infrastructure and produces no telemetry within the victim network.

The LLM provider's side of the interaction is also invisible to the victim. If an adversary is using a frontier model API through a commercial provider, the provider may have visibility into the prompts and responses, but the victim organisation does not. The source does not indicate whether LLM providers are detecting or reporting malicious use of their APIs, and the source is silent on any cooperative detection mechanisms between providers and victim organisations.

From the vantage point of the victim's cloud audit logs, the individual API calls made by ARTEX during its permission probing are legitimate API calls. The DryRun flags and placeholder IDs used during AWS permission probing are standard AWS API parameters. Without correlating multiple calls across a compressed timeframe, the individual events do not indicate malicious activity. The source notes that ARTEX ran an 8-second ARM deployment validation dry-run sweep across 11 Azure resource types. Each individual validation call is a legitimate Azure operation. Only the aggregation of 11 resource type validations in 8 seconds reveals the anomalous pattern.

Limitations

This working paper is subject to several significant limitations.

No telemetry was collected or analysed in the production of this paper. All observations are drawn solely from the source text published by GTIG and Mandiant, supplemented by the provided assessment. No independent verification of the described behaviours was conducted. The specific numbers cited, including 27 events in 23 seconds, 330 to 400 millisecond blob cycles, 8-second ARM sweeps, 491 rotating webhook.site URLs, over 500 web requests, over 1,300 internal email recipients, approximately 400 KB of query results across roughly 20 queries, over 110,000 Active Directory objects, 240 log events and the 15-minute and 4-hour timelines for deep system access, are all reported by the source but have not been reproduced or validated against live data.

The four Sigma rules provided have not been tested against real data. The detection logic is based on behavioural patterns described in narrative form in the source text, not on structured telemetry samples. The rules have not been run against any log repository, SIEM platform or EDR telemetry. Their efficacy, false positive rate and detection coverage are entirely unvalidated. The thresholds

for burst timing, request volume and temporal density are drawn from specific incidents described in the source and may not generalise to other environments or to future ARTEX activity.

The source explicitly warns that these frameworks currently trade stealth for velocity. The detections in this pack are built on the premise that agentic frameworks produce noisy, high-volume, compressed-burst activity patterns. The source states that as the sophistication of the frameworks, the underlying LLMs and the orchestrating human operator increase, GTIG expects to see not only less human interaction but also increased stealth and sophistication. The source's Stage 6 assessment, referenced in the provided assessment as anticipating changes within 1 to 3 months, suggests that the current noisy and iterative patterns may be replaced by more stealthy execution in the near term. Detections built on burst timing and volume will degrade as adversaries improve their operational security and reduce forensic noise.

The artex-probe and ArtexProbe naming conventions are specific to ARTEX and will not catch the other frameworks mentioned in the source, including PentAGI, CAIRN and Hexstrike, or custom agentic harnesses. The source documents that the original ARTEX repository was taken down in October 2026 but forks persist, including ScopeWeaver and ARTEX PRO. These forks may or may not retain the same naming conventions. The source is silent on whether ScopeWeaver or ARTEX PRO use the same probe file and account naming patterns.

webhook.site is a legitimate service used by developers and testers for inspecting HTTP requests. Alerting on all webhook.site traffic will generate false positives. The source documents 491 rotating webhook.site URLs used by ARTEX, but the source does not provide the full list of URLs. Correlation with other behaviours, such as curl pulling scripts and rapid URL rotation, is necessary before alerting. The source is silent on whether other legitimate services could be abused in similar ways by agentic frameworks.

The underlying techniques documented in the source, including Kerberoasting, DCSync, Pass-the-Hash, SQL injection and Pass-the-Ticket, are standard adversary tactics, techniques and procedures. Detections that key on these techniques alone, without incorporating the execution pattern and velocity, will produce overwhelming alert volume from legitimate red team activity, vulnerability scanners and human adversaries. The source notes that multiple sub-agents overwriting one another while pursuing the same objective generated 240 log events with errors. This concurrent agent behaviour is a distinguishing feature, but the source does not provide sufficient detail to determine whether similar patterns appear in legitimate automated testing tools.

This paper is a machine draft produced for a human editor. It has not been reviewed for technical accuracy, detection logic soundness or operational readiness. The Sigma rules referenced have not been validated against any ATT&CK evaluation, MITRE Engage framework or real-world telemetry corpus. The assessment provided alongside the source text is treated as a secondary source of facts, but no facts have been independently corroborated beyond what the source text and assessment establish.

The source text is truncated at Stage 3 of the evolutionary framework. The full description of Stages 4, 5 and 6 is not available in the provided material. The assessment references a Stage 6 assessment with a 1 to 3 month timeframe for anticipated changes, but the source text does not include this content. Any claims about future evolution beyond what the source text explicitly states are drawn from the assessment, not the primary source.

Burst timing thresholds should be tuned per environment. The source provides specific numbers from specific incidents, not universal thresholds. What constitutes anomalous temporal density in

one environment may be normal in another. The source does not provide guidance on threshold selection, environment-specific tuning or baseline establishment. Defenders implementing these detections must conduct their own environmental baselining before deploying alert-type rules based on temporal density.

References

https://cloud.google.com/blog/topics/threat-intelligence/artex-and-friends-enter-agentic-offensive-frameworks/